

Unit 2

Object-Oriented Programming

BT151CO · Python Programming · 10 Hours

Learning Objectives

- Design Python classes with attributes, `__init__`, and methods
- Apply Inheritance to build reusable class hierarchies
- Demonstrate Polymorphism via method overriding and duck typing
- Create and raise Custom Exception classes
- Use Generators and Iterators for memory-efficient processing
- Write reusable functions with `*args` and `**kwargs`
- Organize code into Modules and Packages
 - Theme: Students · BankAccounts · Library Books · Game Characters · Shopping Carts

From Procedural to OOP

Why do we need classes at all?

❌ Procedural Style (Unit 1)

- Data and functions live separately
- 30 students = 30 sets of variables
- Hard to extend — change everywhere
- No natural real-world grouping
- `name = 'Alice'`
- `score = 85`
- `def get_grade(s): ...`

✅ OOP Style (Unit 2)

- Data + behaviour bundled in one class
- One class handles any number of students
- Extend with inheritance — no copy-paste
- Mirrors real-world concepts naturally
- `class Student:`
 - `def __init__(self, name, score): ...`
 - `def get_grade(self): ...`

💡 OOP shines when you have multiple entities of the same kind that each hold their own data and respond to messages.

The Four Pillars of OOP

Pillar	Simple Definition	Python Example
Encapsulation	1. Bundle data + behaviour; 2. hide internals	BankAccount hides balance; exposes deposit()
Inheritance	Child class reuses and extends parent class	SavingsAccount IS A BankAccount
Polymorphism	Same method call → different behaviour	.attack() on Warrior vs Mage vs Archer
Abstraction	Show only what's necessary; hide complexity	account.withdraw(100) hides the maths

 All four principles work together — you rarely use one without the others.

Class vs Object

The blueprint / instance distinction



Class (Blueprint)

- The idea of a Student
- Defines structure: what data & what behavior
- Written once, used many times
- `class Student:`
- `pass` # blueprint



Object (Instance)

- One specific student
- Has its own values for name, score, etc.
- Created from the class at runtime
- `alice = Student()` # object 1
- `bob = Student()` # object 2
- `charlie = Student()` # object 3



Cookie-cutter analogy — Class = cutter shape. Objects = individual cookies stamped out from it.

Class Syntax and `__init__`

```
class Student:
    def __init__(self, name, score):    # constructor
        self.name = name               # instance variable
        self.score = score             # instance variable

# Instantiation – create independent objects
alice = Student("Alice", 85)
bob   = Student("Bob", 72)

print(alice.name)    # Alice
print(bob.name)      # Bob
print(alice.score)   # 85 (bob.score is still 72)
```

- *class* keyword + PascalCase name (PEP 8)
- `__init__` runs automatically on creation
- *self* = this specific object
- *self.name* = name stores value on object
- Each object gets its own independent copy

Understanding `self`

- `self` refers to the specific instance being used
- Always the FIRST parameter of every instance method
- Python passes it automatically — never pass it yourself
- `alice.get_grade()` → Python calls `Student.get_grade(alice)`

```
class Student:
    school = "Aryan"          # class variable
    student_count = 0         # class variable

    def __init__(self, name, score):
        self.name = name      # self = the new object
        self.score = score

    def describe(self):
        # self lets us reach THIS object's data
        print(f"I am {self.name} with score {self.score}")

alice = Student("Alice", 85)
alice.describe()             # self=alice → "I am Alice with score 85"

bob = Student("Bob", 72)
bob.describe()               # self=bob → "I am Bob with score 72"
```

NOTE:

`alice.describe()` ⇔ `Student.describe(alice)`
`bob.describe()` ⇔ `Student.describe(bob)`

So,
inside the first call, **self = alice**, and
Inside the second cell, **self = bob**

self means "the current object using this method right now"

Instance Variables vs Class Variables

	Instance Variable	Class Variable
Where defined	Inside <code>__init__</code> via <code>self.x = ...</code>	Class body, outside <code>__init__</code>
Belongs to	One specific object only	ALL objects of the class
How to access	<code>obj.x</code>	<code>ClassName.x</code> (preferred)
How to modify	<code>self.x = new_val</code>	<code>ClassName.x = new_val</code>
Example	<code>self.name</code> , <code>self.score</code>	<code>Student.school</code> , <code>Student.count</code>

⚠ `alice.school = 'New'` creates an instance variable on `alice` only — it does NOT change `Student.school` for everyone!

Instantiation — Creating Objects

```
alice = Student("Alice", 85)
bob = Student("Bob", 72)
charlie = Student("Charlie", 91)

# Memory model
# alice → [name="Alice", score=85]
# bob → [name="Bob", score=72]
# charlie → [name="Charlie", score=91]
# all share Student.school_name

alice.score = 100          # only alice changes
print(bob.score)          # 72 (bob unchanged)

print(isinstance(alice, Student))  # True
print(isinstance(alice, object))   # True – everything is an object
print(Student.count)              # 3
```

- Call the class like a function → object created
- Python runs `__init__` automatically
- Each object is fully independent
- `isinstance(obj, Class)` checks type
- All objects inherit from object

Instance Methods

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def get_grade(self):                # read self.score
        if self.score ≥ 90: return "A"
        if self.score ≥ 75: return "B"
        return "C"

    def apply_bonus(self, pts):        # modify self.score
        self.score = min(self.score + pts, 100)

    def describe(self):               # calls other methods
        print(f"{self.name}: {self.score} ({self.get_grade()})")

alice = Student("Alice", 85)
alice.describe()                    # Alice: 85 (B)
alice.apply_bonus(10)
alice.describe()                    # Alice: 95 (A)
```

- Functions defined inside a class
- Always receive self as first parameter
- Can read and modify instance variables
- Can call other methods via self.method()

Special (Dunder) Methods

Making objects work with Python's built-in syntax

Method	Triggered by	Purpose
<code>__str__</code>	<code>print(obj)</code> , f-strings	Human-readable string
<code>__repr__</code>	<code>repr(obj)</code> , REPL display	Unambiguous developer string
<code>__len__</code>	<code>len(obj)</code>	Length — e.g. books in a library
<code>__eq__</code>	<code>obj1 == obj2</code>	Value equality
<code>__lt__</code>	<code>obj1 < obj2</code>	Used by <code>sort()</code> , <code>max()</code> , <code>min()</code>
<code>__add__</code>	<code>obj1 + obj2</code>	Custom addition
<code>__contains__</code>	<code>item in obj</code>	Membership test

✓ Always define `__str__` — without it `print(obj)` shows `<__main__.Student object at 0x7f3c...>`



Part 2

Inheritance

Reusing and extending existing classes

- The Is-A Relationship
- `super()` and Method Overriding
- Method Resolution Order (MRO)

Inheritance — The Is-A Relationship

```
class BankAccount:                                # PARENT
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

class SavingsAccount(BankAccount):                # CHILD – inherits everything
    def __init__(self, owner, balance, rate):
        super().__init__(owner, balance) # parent sets owner, balance
        self.interest_rate = rate        # NEW attribute

    def apply_interest(self):                  # NEW method
        self.balance += self.balance * self.interest_rate

acct = SavingsAccount("Alice", 10000, 0.06)
acct.deposit(5000)                             # ← inherited from BankAccount, not redefined
acct.apply_interest()                          # ← unique to SavingsAccount
```

- SavingsAccount **IS A** BankAccount
- GraduateStudent **IS A** Student
- Archer **IS A** GameCharacter
- Child gets ALL parent attrs + methods free
- Child adds new attrs / overrides methods

`super()` and Method Overriding

```
class LoanAccount(BankAccount):
    def __init__(self, owner, credit_limit):
        super().__init__(owner, balance=0) # ← ALWAYS call this first
        self.credit_limit = credit_limit

    def withdraw(self, amount):
        # OVERRIDE parent's withdraw
        if self.balance - amount < -self.credit_limit:
            raise ValueError("Exceeds credit limit")
        self.balance -= amount
        # loan can go below zero

    def __str__(self):
        base = super().__str__()
        # REUSE parent's string
        return f"{base} | Limit: {self.credit_limit}"

# SavingsAccount uses inherited deposit() unchanged
# LoanAccount uses its OWN withdraw() – overriding the parent
```

- `super()` safely accesses the parent class
- `super().__init__()` first → parent sets up attrs
- Override = redefine method in child → child wins
- Use `super().method()` to extend, not replace

Method Resolution Order (MRO)

Which version of a method does Python use?

```
class Person:
    def greet(self):
        print("Hello from Person")

class Student(Person):
    def greet(self):
        print("Hello from Student")

class GraduateStudent(Student):
    pass          # no greet – uses nearest ancestor

g = GraduateStudent()
g.greet()      # "Hello from Student" ← Student before Person

print(GraduateStudent.__mro__)
# (<class 'GraduateStudent'>, <class 'Student'>,
# <class 'Person'>, <class 'object'>)

# Multiple inheritance
class Duck(Flyable, Swimmable):
    pass

print(Duck.__mro__)  # Duck → Flyable → Swimmable → object
```

- Python searches classes left-to-right in `__mro__`
- First class that has the method wins
- GraduateStudent → Student → Person → object
- `super()` always follows the MRO automatically
- `Class.__mro__` shows the exact order which Python uses.



Part 3

Polymorphism

Same interface — different behaviour

- Method Overriding in Action
- Duck Typing
- Operator Overloading

Polymorphism — Method Overriding in Action

```
class Warrior(GameCharacter):
    def attack(self):
        print(f"{self.name} swings axe for {25*self.level} damage!")

class Mage(GameCharacter):
    def attack(self):
        print(f"{self.name} casts fireball for {40*self.level} magic dmg!")

class Archer(GameCharacter):
    def attack(self):
        print(f"{self.name} fires an arrow for {20*self.level} damage!")

party = [Warrior("Thor",5), Mage("Merlin",8), Archer("Legolas",6)]

for character in party:          # ← SAME call for all three
    character.attack()
# Thor   swings axe for 125 damage!
# Merlin casts fireball for 320 magic dmg!
# Legolas fires an arrow for 120 damage!
```

- Same method name, completely different output
- Python decides at runtime which version to call
- The loop doesn't care what type each object is
- Add new character types → loop needs NO changes

Duck Typing

"If it walks like a duck and quacks like a duck, it's a duck."

```
class PDFReport:
    def generate(self): return "report.pdf"

class ExcelReport:
    def generate(self): return "report.xlsx"

class HTMLReport:
    def generate(self): return "report.html"

# These three share NO parent class (beyond object)
def run_report(generator):          # works with any object
    filename = generator.generate() # that has generate()
    print(f"Saved: {filename}")

run_report(PDFReport())    # ✓ Saved: report.pdf
run_report(ExcelReport()) # ✓ Saved: report.xlsx
run_report(HTMLReport())  # ✓ Saved: report.html
```

- Python checks if the method EXISTS — not the type
- Any object with generate() works in run_report()
- No shared parent class required
- Errors appear at runtime if the method is missing

- Contrast with Java/C++ where a formal interface is required. Python doesn't need a formal contract — if the method exists, it works.
- This is why Python is described as a 'permissive' OOP language.

Operator Overloading

```
from functools import total_ordering

@total_ordering
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score
    def __str__(self): return f"{self.name} ({self.score})"
    def __eq__(self, o): return self.score == o.score
    def __lt__(self, o): return self.score < o.score # enables sort

students = [Student("Alice",85), Student("Bob",72), Student("Charlie",91)]

students.sort()           # uses __lt__
top = max(students)       # uses __lt__
print(top)                # Charlie (91)

# Without __eq__, == compares memory addresses (always False!)
a = Student("Alice", 85)
b = Student("Alice", 85)
print(a == b) # True (compares scores thanks to __eq__)
```

- `__add__` → + | `__sub__` → -
- `__eq__` → == | `__lt__` → <
- `__len__` → len() | `__str__` → print()
- `@total_ordering` generates `__le__`, `__gt__`, `__ge__`, if you provide `__eq__`, and `__lt__`



Part 4

Exceptions

Reporting and handling errors the OOP way

- Python Exception Hierarchy
- Custom Exception Classes
- try / except / else / finally

Python's Exception Hierarchy

```
# Hierarchy (simplified)
BaseException
├── SystemExit          # sys.exit()
├── KeyboardInterrupt  # Ctrl+C
├── Exception           # ← catch THIS for user-facing errors
│   ├── ValueError     # int("hello")
│   ├── TypeError      # 85 + "hello"
│   ├── NameError      # using undefined variable
│   ├── IndexError     # list[100] out of range
│   ├── KeyError       # dict["missing"]
│   ├── FileNotFoundError
│   └── ZeroDivisionError

# Catch most specific FIRST
try:
    x = int("hello")
except ValueError:
    print("Not a number")    # ← caught here
except Exception as e:
    print(f"Other: {e}")    # only if not ValueError
```

- All exceptions are classes that inherit from **BaseException**
- Catch Exception (not **BaseException**) for user errors
- Most specific except clause must come FIRST
- except ValueError before except Exception
 - Because **ValueError** IS-A Exception
- In Python:
 - almost everything is an **Exception** object
 - even many things called "errors" are still exceptions.
- Some OOP languages separate Errors and Exceptions:

Error	Exception
A serious problem that usually should not (or cannot) be handled	A problem your program is expected to handle

Custom Exception Classes

```
class BankError(Exception):
    # Base for all bank-related errors
    pass

class InsufficientFundsError(BankError):
    def __init__(self, requested, available):
        super().__init__(
            f"Need {requested:.2f}, have {available:.2f}"
        )
        self.requested = requested    # structured data for callers
        self.available = available

class BankAccount:
    def __init__(self, acc_no, balance):
        self.acc_no = acc_no
        self.balance = balance

    def withdraw(self, amount):
        # check if the account has sufficient balance or not
        if amount > self.balance:
            raise InsufficiendFundsError(amount, self.balance)

        # Otherwise perform withdrawl
        self.balance = self.balance - amount
```

- Inherit from Exception (never BaseException)
- Create a base class per module (BankError)
- Store structured data as attributes
- Name as XxxError — InsufficiendFundsError

`try` / `except` / `else` / `finally`

```
try:
    account.withdraw(1000)

except InsufficientFundsError as e:    # most specific
    print(f"Not enough: {e.available:.2f} available")

except AccountFrozenError as e:
    print(f"Account {e.acct_no} is frozen – call the bank")

except BankError as e:                # any other bank error
    print(f"Bank error: {e}")

else:
    print("Withdrawal successful! ) # only if NO exception

finally:
    log_transaction(account)           # ALWAYS executed
```

- try — code that might fail
- except — handle specific exception (most specific first)
- else — runs ONLY if try succeeded (no exception)
- finally — ALWAYS runs — use for cleanup (close files, DB)



Part 5

Generators & Iterators

Memory-efficient data processing

- The Iterator Protocol (`__iter__` + `__next__`)
- Generators and `yield`

Iterators — The Protocol

```
# What Python DOES for: for x in [1, 2, 3]
it = iter([1, 2, 3])    # calls list.__iter__()
print(next(it))         # 1 - calls it.__next__()
print(next(it))         # 2
print(next(it))         # 3
next(it)                # StopIteration ← loop stops

# Custom iterator - BookShelf
class BookShelf:
    def __init__(self):
        self._books = []

    def add(self, book):
        self._books.append(book)

    def __iter__(self):
        return iter(self._books)

shelf = BookShelf()
shelf.add("Python Basics")
shelf.add("OOP Concepts")

for book in shelf:      # uses __iter__
    print(book)
# Python Basics
# OOP Concepts
```

- Iterable = any object you can loop over
- Iterator = iterable with `__iter__` + `__next__`
 - the thing that actually gives values one-by-one during the loop
- `__next__()` returns next value or raises `StopIteration`
- for loops use `iter()` and `next()` internally

Generators and `yield` — Lazy Evaluation

```
def countdown(n):
    while n > 0:
        yield n          # pause, return n, resume on next next()
        n -= 1

for num in countdown(5):
    print(num, end=" ")  # 5 4 3 2 1

# — Memory comparison —————
# List: computes ALL 1M values now → uses ~8 MB
squares_list = [x**2 for x in range(1_000_000)]

# Generator: computes values ONE AT A TIME → uses ~100 bytes
squares_gen = (x**2 for x in range(1_000_000))

for sq in squares_gen:
    pass                # loop works exactly the same
                        # but memory stays tiny!
```

- yield = pause, return value, resume from here next call
- def + yield → generator function (not a regular function)
- Generator expression: (x for x in ...) uses ~100 bytes
- List comprehension: [x for x in ...] loads ALL values now
- Lists vs Generators
 - Lists: when data is small and reused often
 - Generators: when data is huge or streamed lazily
- Most Important Concept:
 - "Compute only when needed"

`\*args` and `\*\*kwargs` — Variable Arguments

```
def average(*scores):          # *args – any number of args
    return sum(scores) / len(scores) if scores else 0

average(85, 72, 91)           # 82.67
average(70, 80, 90, 75, 65)   # 76.0

def profile(name, **info):     # **kwargs – any keywords
    print(f"Profile: {name}")
    for k, v in info.items():
        print(f"  {k}: {v}")

profile("Alice", gpa=3.75, major="CS", scholarship=True)
# Profile: Alice
#   gpa: 3.75
#   major: CS
#   scholarship: True

# Full parameter order rule:
def full(pos1, pos2, *args, kw_only=0, **kwargs): ...
```

- `*args` → collects positional args → tuple
- `**kwargs` → collects keyword args → dict
- Order: regular → `*args` → keyword-only → `**kwargs`
- Unpack with: `func(*my_list)` or `func(**my_dict)`

Scope — The LEGB Rule

In what order does Python search for a name?

Scope	Location	Example
L — Local	Inside the current function	score = 85 inside get_grade()
E — Enclosing	Surrounding function (nested funcs)	count in make_counter() function
G — Global	Module (file) level	PASSING_SCORE = 60 at top of file
B — Built-in	Python's built-in names	print, len, range, type, ...

⚠ Use global / nonlocal with care. Prefer passing values as parameters and returning results — it makes code easier to test.

Scope — The LEGB Rule

In what order does Python search for a name?

```
x = "GLOBAL X" # Global scope

def outer():
    x = "ENCLOSING X" # Enclosing scope

    def inner():
        x = "LOCAL X" # Local scope

        # Local
        print("1.", x)

        # Enclosing
        def show_enclosing():
            print("2.", x)

        show_enclosing()

    inner()
    print("3.", x)

outer()

print("4.", x)

# Built-in example
numbers = [1, 2, 3]
print("5.", len(numbers))
```

- Module = a .py file | Package = folder + __init__.py
- import module → use module.name to access names
- from module import X → use X directly
- if __name__ == '__main__': blocks run on import

Output

```
1. LOCAL X
2. LOCAL X
3. ENCLOSING X
4. GLOBAL X
5. 3
```

Modules and Imports

Organising reusable code into .py files

```
# Four import styles
import student_utils                        # full module
from student_utils import GradeBook        # specific class
import student_utils as su                 # alias
from student_utils import *                # ⚠ avoid in production

# The __name__ guard - runs ONLY when executed directly
# student_utils.py
def calculate_gpa(scores):
    return sum(scores) / len(scores)

if __name__ == "__main__":                 # NOT executed on import
    print(calculate_gpa([85, 90, 72]))     # 82.33

# Package structure
# student_system/
# |— __init__.py      ← makes it a package
# |— students.py
# |— courses.py
# |— exceptions.py
```

- Module = a .py file | Package = folder + `__init__.py`
- `import module` → use `module.name` to access names
- `from module import X` → use `X` directly
- `if __name__ == '__main__':` blocks run on import

Unit 2 — Summary

Key concepts at a glance

Classes, Objects & Inheritance

- Class = blueprint · Object = instance
- `__init__` sets up state · `self` = this object
- Instance vars (per object) vs class vars (shared)
- Child IS A parent · `super().__init__()` first
- Method overriding · MRO defines search order
- `isinstance(obj, Class)` checks the hierarchy

Polymorphism, Exceptions, Generators & Modules

- Same call → different behaviour (polymorphism)
- Duck typing: check method exists, not the type
- Custom exceptions inherit from `Exception`
- `try / except / else / finally` pattern
- `yield` = lazy values on demand (generators)
- `*args` → tuple · `**kwargs` → dict · LEGB scope

 Next — Unit 3: `@property`, `@abstractmethod`, `@classmethod`, Composition, and Design Patterns.